

# Web Systems Development

**RPI**



# Pull the Course Repo, Then Start the Pantry Example

Begin at the parent directory that contains the ITWS2110-course repository.

BEFORE THE DEMO

1

GET THE LATEST COURSE FILES

## Update ITWS2110-course

```
% cd ITWS2110-course
% git pull
```

Enter the course repository, then pull the latest examples and Compose configuration.

THEN



2

BUILD AND WATCH THE EXAMPLE

## Enter wk3/pantry and start

```
% cd wk3/pantry
% docker compose watch
```

Compose must run from wk3/pantry so it can find the example's compose.yaml file.

**Leave docker compose watch running while we work in the pantry example.**

Docker Desktop must be running.

If git pull reports local conflicts, stop and ask before overwriting your work.

# Where We Are

- **Core engineering pillars, in order**
  - Docker — done
  - Testing — this week.
- **What ties it together**
  - Automated tests are how homework is checked — so you learn to write them first
  - Every tool this week runs inside Docker.

# Readings — Testing

- **Chojrin, Modern Testing with PHP — chapters 1, 2, 7** ([RPI library](#))
  - 1 — what a test is for; 2 — your first PHPUnit test; 7 — the Grocy case study, which is Homework 2
- **Playwright — Writing tests** ([playwright.dev](#))
  - A robot that uses your site. Read it; we run it in class.
- **Ian Cooper — TDD, Where Did It All Go Wrong** ([video, 1 hr](#))
  - Most people test the wrong unit. This is the talk I will argue with today.
- **Vocke — The Practical Test Pyramid (three sections)**
  - Why many fast tests and few slow ones is not a matter of taste

# What is software testing?



# Software Testing

*“Software testing is the process of evaluating and verifying that a software product or application does what it’s supposed to do.”*

<https://www.ibm.com/think/topics/software-testing>

**If all of the tests passed, does it mean there are no software bugs?**

**If all of the tests passed, does it mean there are no software bugs?**

NO. It just means that any bugs have not been thought of by the test writer.



# Why Testing Matters

- Ensures reliability
- Prevents regressions
- Improves code quality
- Enables faster, reliable development
- Without testing, can't make changes without breaking things

# Automated Testing & CI/CD

- **CI = Continuous Integration**

- Every push triggers the tests
- Homework 2 ships a GitHub Actions workflow: Grocy on a runner, all three suites
- Task 5: make it red once on purpose, then fix it

- **CD = Continuous Deployment**

- If tests pass, deploy automatically; if they fail, block it

- **Why this matters**

- Manual testing does not scale
- Tests are a safety net — not a replacement for good design

# Test-Driven Development

**Test-Driven Development (TDD) is the practice of working in a structured cycle where writing tests comes before writing production code.** The process involves three steps, sometimes called the *red-green-refactor cycle*:

1. **Write a failing test**
2. **Make the test pass by writing just enough production code**
3. **Refactor the production code to meet your quality standards**

Particularly Important in the age of AI

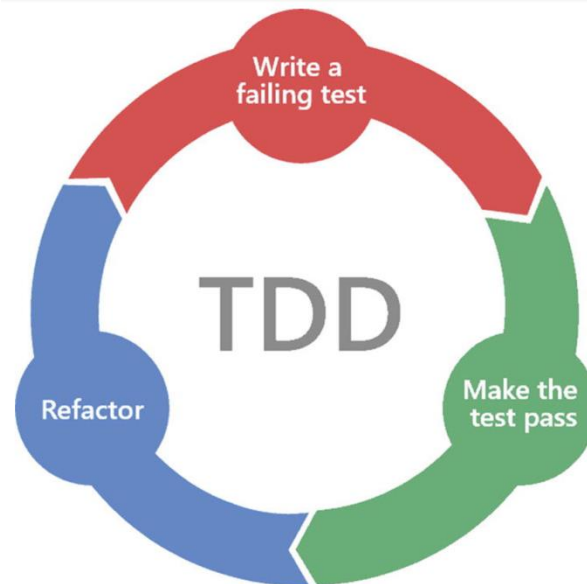
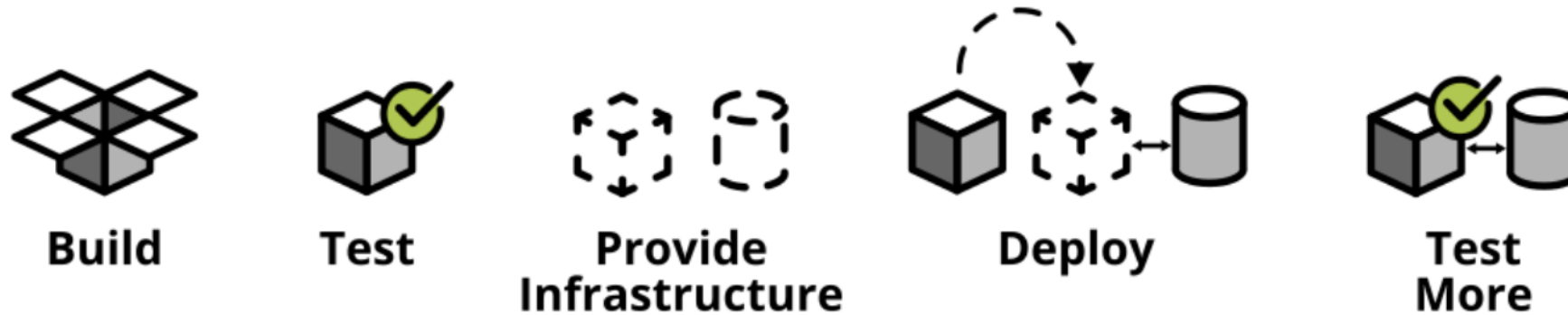


Figure 5-1 TDD cycle diagram

# Testing Functionality and Deployments



*Figure 1: Use build pipelines to automatically and reliably get your software into production*

# Types of Tests

*The first distinction between them is the abstraction level, which is in turn determined by the specific subject under test.*

## **Unit tests**

Unit tests are the most atomic kind of tests you can think of: they are meant to assess the correctness of a single piece of software within a system, typically a class or a function.

## **Acceptance tests**

Acceptance tests deal with the whole code base as a single piece. Typically, these tests simulate the user input and any other external dependencies and make assertions about the system's response before it is sent back to the user.

## **End-to-end tests**

One level closer to reality are the end-to-end tests which aim to interact with the system in a way that is indistinguishable from actual user activity.

## **Integration tests**

Integration tests are tasked with making sure that the system can connect to its satellite components (databases, cache systems, etc.).

## **Smoke tests**

Finally, smoke tests are very shallow tests that are meant to be used as early alarm systems. In other words, all they assess is whether the system is alive, regardless of its correct behavior.

# Writing Good Tests

There are a few traits every good test features:

## RELIABLE TO RUN

### Isolated

They produce the same results no matter the order in which they are executed.

### Composable

They complement each other.

### Repeatable

They can be run as many times as needed without any additional effort.

### Fast

They produce feedback quickly.

### Readable

They are succinct and easy to understand, expressing clearly what they are verifying.

## MEANINGFUL WHEN IT SPEAKS

### Behavioral

Their result changes as a result of changes in the behavior of the code under test.

### Structure-insensitive

Their result doesn't change as a result of changes in the structure of the code under test.

### Specific

When they fail, the reason for the failure is obvious.

### Deterministic

Their result is the same if nothing changes.

### Predictive

Their passing constitutes a strong indicator that the code under test is ready to go to production.

# The Testing Pyramid

If you want to get serious about automated tests for your software there is one key concept you should know about: the **test pyramid**. Mike Cohn came up with this concept in his book *Succeeding with Agile*. It's a great visual metaphor telling you to think about different layers of testing. It also tells you how much testing to do on each layer.

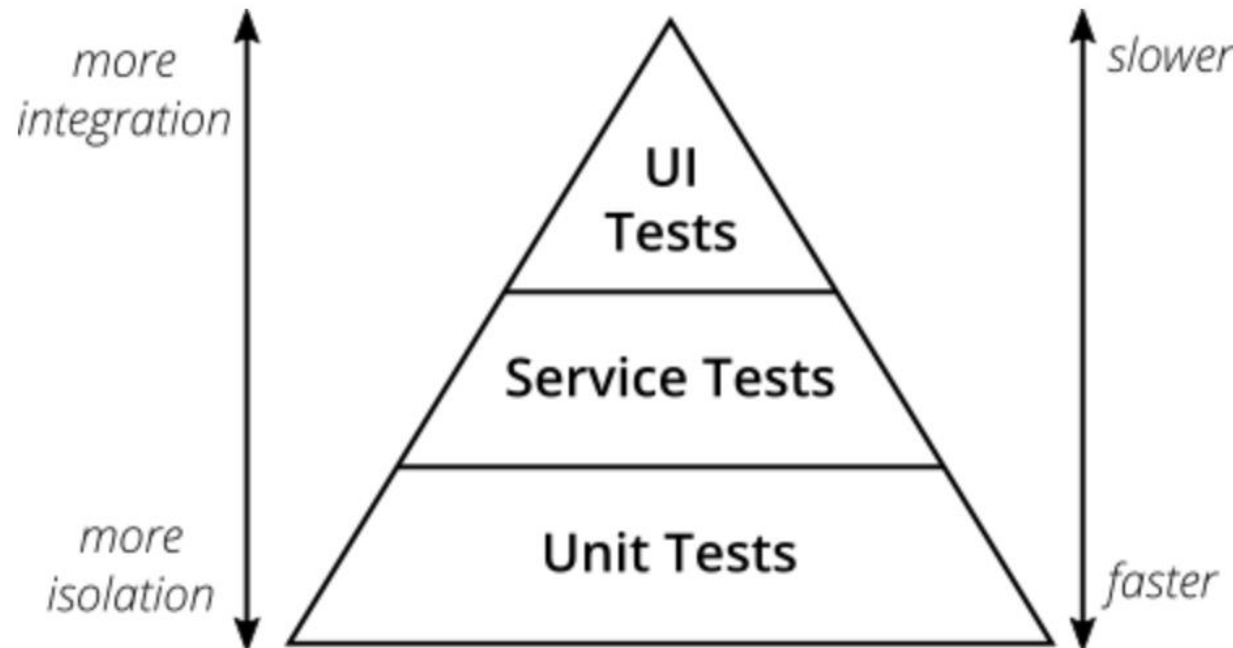


Figure 2: The Test Pyramid

# Pantry App

## Pantry

Today is 2026-09-08. Products, how much is on the shelf, and when it goes off. The same data is at </api/products>.

Name  Best before  [Add product](#)

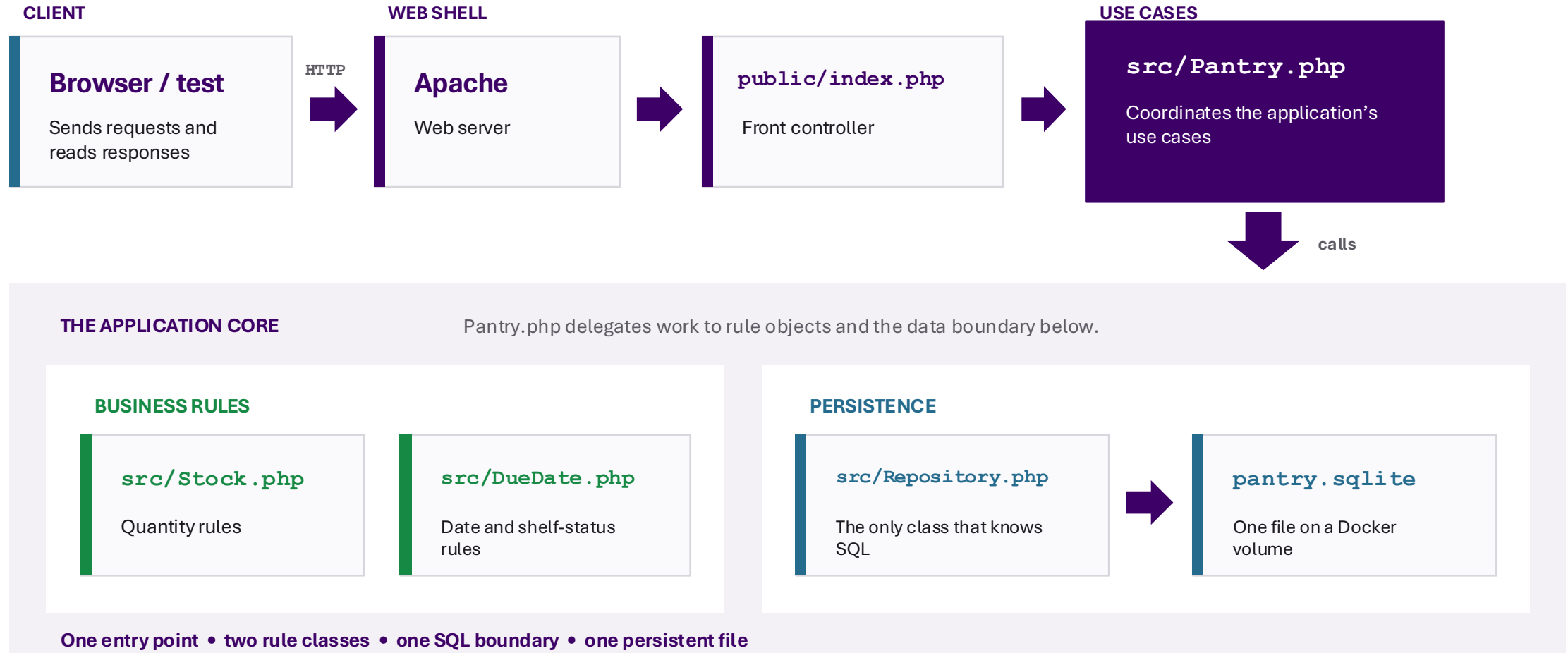
| Product | In stock | Best before |       | Buy                            |                     | Use                                                |
|---------|----------|-------------|-------|--------------------------------|---------------------|----------------------------------------------------|
| Milk    | 0        | 2026-09-18  | fresh | <input type="text" value="1"/> | <a href="#">Buy</a> | <input type="text" value="1"/> <a href="#">Use</a> |

What are some “rules” of how a pantry app should function?



# A Thin Web Shell Routes Requests into the Domain

Browser-facing code stays small; Pantry coordinates use cases; domain classes hold the rules; one repository owns every SQL statement.



# Project Structure — inclass/wk3/pantry

|                    |                                                             |                                      |
|--------------------|-------------------------------------------------------------|--------------------------------------|
| src/Stock.php      | add(), consume()                                            | pure PHP, unit tested                |
| src/DueDate.php    | daysLeft(), status()                                        | expired / due soon / fresh           |
| src/Pantry.php     | the use cases -- the page and the API both call it          |                                      |
| src/Repository.php | the only file that knows SQLite exists                      |                                      |
| public/index.php   | /api/... returns JSON; everything else is the page          |                                      |
|                    |                                                             |                                      |
| tests/unit/        | StockTest.php, DueDateTest.php                              | PHPUnit                              |
| tests/api/         | products.spec.js                                            | Playwright request                   |
| tests/e2e/         | pantry.spec.js                                              | Playwright + Chromium                |
|                    |                                                             |                                      |
| docker/apache.conf | DocumentRoot public/ + FallbackResource /index.php          |                                      |
| Dockerfile         | Playwright image + Apache + PHP 8.3 + Composer -- ONE image |                                      |
| docker-compose.yml | web · unit · api · e2e                                      |                                      |
|                    |                                                             |                                      |
| 0-run-the-app.md   | 1-phpunit.md                                                | 2-prompt.md 3-api.md 4-playwright.md |

# What the Test Suite Specifies

Read the test names as behavioral rules: the left column defines date status; the right defines inventory behavior.

## Due Date

12 behaviors

- Days left counts forward to the date
- Days left is zero on the day
- Days left goes negative once the date has passed
- Days left crosses month boundaries
- A product with no date has no status
- Shelf label: data set “last week”
- Shelf label: data set “yesterday”
- Shelf label: data set “today”
- Shelf label: data set “in five days”
- Shelf label: data set “in six days”
- Shelf label: data set “next year”
- The warning window can be changed

## Stock

9 behaviors

- A new stock is empty
- A stock can start with an amount
- A stock cannot start below zero
- Buying adds to the amount
- Buying nothing is refused
- Buying a negative amount is refused
- Using some takes it off the shelf
- Using all of it leaves zero
- Using a negative amount is refused

**PROMPT: Consider the Pantry app shown and potential functionality and list of unit tests. Create a table that describes additional unit tests.**

**Pantry**

Today is 2026-09-08. Products, how much is on the shelf, and when it goes off. The same data is at [/api/products](#).

Name

Best before

☐

Add product

| Product | In stock | Best before |       | Buy                            |     | Use                            |     |
|---------|----------|-------------|-------|--------------------------------|-----|--------------------------------|-----|
| Milk    | 1        | 2026-09-18  | fresh | <input type="text" value="1"/> | Buy | <input type="text" value="1"/> | Use |

# Additional Unit Tests — Due Date

These cases extend the existing suite without duplicating it.

## DUE DATE

| ADDITIONAL UNIT TEST                                           | EXPECTED RESULT                                                        |
|----------------------------------------------------------------|------------------------------------------------------------------------|
| Days left crosses a year boundary                              | The difference is calculated correctly between December and January    |
| Days left handles a leap day                                   | February 29 is included correctly in leap years                        |
| Days left is unaffected by the time of day                     | The calculation uses calendar dates and does not change during the day |
| Days left is unaffected by daylight-saving changes             | Crossing a clock change does not add or remove a day                   |
| The warning label changes at the exact warning-window boundary | A product exactly at the configured limit receives the intended label  |
| A product one day outside the warning window is fresh          | The warning label is not applied prematurely                           |
| Changing the warning window reclassifies existing products     | Status is recalculated using the new window                            |
| A warning window of zero is supported                          | Only products due today receive the warning status                     |
| A negative warning window is refused                           | Invalid configuration is rejected                                      |
| An impossible date is refused                                  | Invalid dates such as February 30 cannot be stored                     |

# Additional Unit Tests — Product

These cases extend the existing suite without duplicating it.

## PRODUCT

| ADDITIONAL UNIT TEST                                | EXPECTED RESULT                                                                        |
|-----------------------------------------------------|----------------------------------------------------------------------------------------|
| A product can be created without a best-before date | The product is created and has no shelf status                                         |
| A product can be created with a valid name and date | Its name, date, and initial stock are stored correctly                                 |
| A blank product name is refused                     | No unnamed product is created                                                          |
| A whitespace-only product name is refused           | Whitespace does not count as a valid name                                              |
| Leading and trailing spaces are removed from a name | The normalized product name is stored                                                  |
| Duplicate product names are handled consistently    | The app either rejects duplicates or creates a distinct product, according to its rule |
| A new product starts with zero stock                | Adding a product through the product service produces an empty stock                   |

## PRODUCT LIST

| ADDITIONAL UNIT TEST                              | EXPECTED RESULT                                     |
|---------------------------------------------------|-----------------------------------------------------|
| Products are returned in the intended order       | The list follows the defined ordering rule          |
| An empty pantry returns an empty list             | No placeholder or invalid product is returned       |
| Each product's status is calculated independently | One product's date or stock does not affect another |

# Additional Unit Tests — Stock

These cases extend the existing suite without duplicating it.

## STOCK

| ADDITIONAL UNIT TEST                                         | EXPECTED RESULT                                                 |
|--------------------------------------------------------------|-----------------------------------------------------------------|
| Using nothing is refused                                     | Stock remains unchanged                                         |
| Using more than is available is refused                      | Stock cannot fall below zero                                    |
| A refused purchase leaves the amount unchanged               | Invalid operations do not partially update stock                |
| A refused use leaves the amount unchanged                    | The original amount is preserved                                |
| Multiple purchases accumulate correctly                      | The final amount equals the starting amount plus all purchases  |
| Multiple uses subtract correctly                             | The final amount reflects every valid use                       |
| Buying after stock reaches zero works                        | The purchased amount becomes the new stock level                |
| Fractional amounts are handled according to the stock policy | Fractions are either accepted accurately or rejected explicitly |
| Non-numeric amounts are refused                              | The stock value remains unchanged                               |
| Extremely large amounts are handled safely                   | The amount does not overflow or become corrupted                |

# Additional Unit Tests — Service & Persistence

These cases extend the existing suite without duplicating it.

## API / SERVICE

| ADDITIONAL UNIT TEST                                    | EXPECTED RESULT                                              |
|---------------------------------------------------------|--------------------------------------------------------------|
| A newly added product appears in the product collection | The returned representation contains the saved product       |
| Buying updates the returned stock amount                | Subsequent reads expose the new quantity                     |
| Using stock updates the returned stock amount           | Subsequent reads expose the reduced quantity                 |
| Invalid product input produces a validation error       | Bad input does not create or modify a product                |
| An unknown product identifier is handled safely         | The operation returns a not-found result and changes nothing |

## PERSISTENCE

| ADDITIONAL UNIT TEST                          | EXPECTED RESULT                                          |
|-----------------------------------------------|----------------------------------------------------------|
| Product data survives a save-and-reload cycle | Name, stock, and best-before date are restored unchanged |



# How many unit tests are too much?



# There Is No Magic Number of Unit Tests

You have too many when another test adds maintenance cost—but no new confidence about behavior or risk.

## KEEP OR ADD A TEST WHEN IT...

1

### Covers a distinct behavior

A new rule, boundary, error, or important scenario.

2

### Protects a meaningful risk

Failure would affect users, money, data, or a contract.

3

### Produces useful diagnosis

Its failure identifies what stopped being true.

## CONSOLIDATE OR REMOVE WHEN IT...

1

### Repeats the same evidence

Several tests prove the same behavior with no new boundary.

2

### Tests implementation details

A harmless refactor breaks it while behavior stays correct.

3

### Costs more than it teaches

It is slow, flaky, opaque, or expensive to maintain.

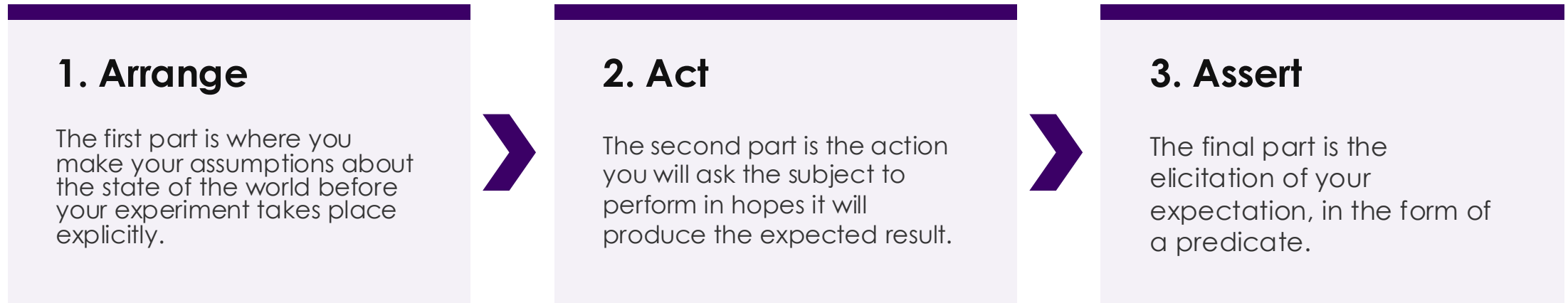
## THE BETTER QUESTION

## What unique behavior or risk does this test prove?

If the answer is “none,” the test is probably redundant—not reassuring.

# Arrange, Act, Assert

In general, every test can be thought of as a three-act play that follows this pattern:



**Example — append a letter to the end of a string**

```
# Arrange
original = "Tes"
# Act
result = appendLetter(original, "t")
# Assert
assert result == "Test"
```

# Testing Tools This Semester

- PHPUnit — unit tests on PHP. The book's tool, Laravel's tool.
- Playwright — browser tests and API tests. Language-agnostic: it does not know your server is PHP.
- Both run inside Docker containers. Nothing installed on your laptop.
- (Pest is PHPUnit with a friendlier syntax on top. We use PHPUnit here so it matches the readings. )

# Installing PHPUnit with Composer

Keep the test runner inside the project—not installed globally—so every project carries its own compatible version.

## THREE WAYS TO GET THE CODE

### ZIP

Download and manage the files yourself.

### PHAR

Download one executable and manage its path.

### COMPOSER

Declare a project dependency and let Composer manage it.

### OUR CHOICE

**Composer**  
**Project-local**

## 1 ADD THE DEVELOPMENT DEPENDENCY

```
composer require --dev phpunit/phpunit
```

# Unit Testing (PHPUnit)

Fast feedback from one class—no server, database, or network.

tests/unit/StockTest.php

```
public function test_buying_adds_to_the_amount(): void
{
    $stock = new Stock(2);           // arrange

    $stock->add(3);                   // act

    $this->assertSame(5.0, $stock->amount()); // assert
}
```

FAST • ISOLATED • NO I/O

## READ THE THREE ACTS

1

### ARRANGE

Create a Stock with 2 items.

2

### ACT

Add 3 more items.

3

### ASSERT

The amount must now be 5.0.

## A TEST NAME IS A SENTENCE THAT SHOULD BE TRUE

Read nine method names and you have a behavioral specification—without opening Stock.php.

## WHY THE BLANK LINES?

They make Arrange → Act → Assert visible before you read the details.

# Writing PHPUnit Test Cases

A good test name reads like a requirement—and a failure tells you exactly what behavior broke.

1

## Match the subject

Survey.php → SurveyTest.php

2

## Make the method discoverable

Start the public method with `test_` or add `#[Test]`.

3

## Name the behavior and outcome

`test_average_rounds_to_one_decimal_place`

4

## Keep one behavior per test

Arrange, act, assert—keep setup small and local.

5

## Use precise assertions

Prefer `assertSame($expected, $actual)` when types matter.

**AVOID**

`testSomething()` and tests that verify several unrelated behaviors.

```
<?php
declare(strict_types=1);
use PHPUnit\Framework\TestCase;
class MyTestCase extends TestCase
{
    public function testSomething()
    {
        // This is a test
    }
    /**
     * @test
     */
    public function something()
    {
        // This is a test
    }
    #[Test]
    public function somethingElse()
    {
        // This is a test
    }
    public function no_test()
    {
        // This is not a test
    }
}
```

# PHPUnit Discovers Tests Two Ways

The method name starts with test—or the method carries #[Test]. Choose one convention and use it consistently.

## OUR EXAMPLES

### 1. The test\_ prefix

```
public function
test_buying_adds_to_the_amount(): void
{
    // arrange, act, assert
}
```

Because the public method begins with test, PHPUnit finds it. No attribute is needed.

#### --testdox turns either method name into a sentence

```
Buying adds to the amount    •    Should calculate three plus
five
```

## THE BOOK'S STYLE

### 2. The #[Test] attribute

```
use PHPUnit\Framework\Attributes\Test;

#[Test]
public function
should_calculate_three_plus_five(): void
{
    // arrange, act, assert
}
```

The attribute marks the method as a test, so the method name can begin with should\_.

#### OLDER TUTORIAL? AVOID /\*\* @test \*/

Annotations are deprecated in PHPUnit 11 and removed in PHPUnit 12. Use #[Test] instead.

Course convention: use the test\_ prefix in our codebase. Both styles are valid in PHPUnit 10 and 11.



# Calculate

- Can combine some tests
- Sometimes better separate.

```
public function should_calculate_X_plus_Y(): void {  
    $calculator = new Calculator();  
    $this->assertEquals(8, $calculator->add(3, 5));  
    $this->assertEquals(8, $calculator->add(4, 4));  
    $this->assertEquals(3, $calculator->add(2, 1));  
}
```

```
<?php  
declare(strict_types=1);  
use App\Calculator;  
use PHPUnit\Framework\Attributes\DataProvider;  
use PHPUnit\Framework\TestCase;  
use PHPUnit\Framework\Attributes\Test;  
use PHPUnit\Framework\Attributes\Group;  
class CalculatorTest extends TestCase  
{  
    #[Test]  
    public function should_calculate_three_plus_five(): void {  
        $calculator = new Calculator();  
        $this->assertEquals(8, $calculator->add(3, 5));  
    }  
    #[Test]  
    public function should_calculate_four_plus_four(): void {  
        $calculator = new Calculator();  
        $this->assertEquals(8, $calculator->add(4, 4));  
    }  
    #[Test]  
    public function should_calculate_two_plus_one(): void {  
        $calculator = new Calculator();  
        $this->assertEquals(3, $calculator->add(2, 1));  
    }  
}
```

# For Stock, assertEquals Protects the Type Contract

The choice is not cosmetic: one checks a value; the other checks the value and its PHP type.

## THE BOOK'S CHOICE

### assertEquals

==

```
$this->assertEquals(8, 8.0);
```

**PASSES** — numerically equal

Useful when the value is what matters and int versus float is irrelevant.

## OUR STOCK TESTS

### assertSame

===

```
$this->assertSame(5.0, 5);
```

**FAILS** — float expected, int received

It verifies both the value 5.0 and the float contract promised by Stock.

**KEEP assertEquals FOR STOCK**

A passing test should prove amount() returned the right number in the right type—not merely something PHP considers equal.

Source: PHPUnit manual, Assertions. Scalar example shown: `8 == 8.0` is true; `5.0 === 5` is false.

# One Command Runs PHPUnit in a Disposable Container

The shell command selects the Compose service; the service definition supplies the PHPUnit command.

RUN THIS IN THE TERMINAL

```
% docker compose run --rm unit
```

Docker Compose resolves the service named unit



WHAT COMPOSE SEES

```
unit:
  <<: *app
  profiles: [test]
  command: ./vendor/bin/phpunit --testdox
```

WHAT HAPPENS

1

## Start unit

Compose creates a one-off container using the shared \*app configuration.

2

## Run PHPUnit

The service executes ./vendor/bin/phpunit inside the project container.

3

## Print TestDox

--testdox displays each discovered test as a readable sentence.

run = one-off container • --rm = delete it afterward • unit = service name • --testdox = sentence-style output

Because unit is explicitly targeted, Compose runs the profile-gated test service without starting the web application.

# Red First — Reading a Failure

Change `+=` to `-=` in `Stock::add()` and run the suite:

```
.....F.....                21 / 21 (100%)  
  
1) StockTest::test_buying_adds_to_the_amount  
Failed asserting that -1.0 is identical to 5.0.  
/app/tests/unit/StockTest.php:52
```

- Which sentence stopped being true → what it expected and what it got → which line. **You found the bug without reading the code.**
- Green means every sentence written is true. **It does not mean every sentence was written.** Nine tests pass — and the rice went to minus three.

# Part 2 — Write the Prompt, Not the Test

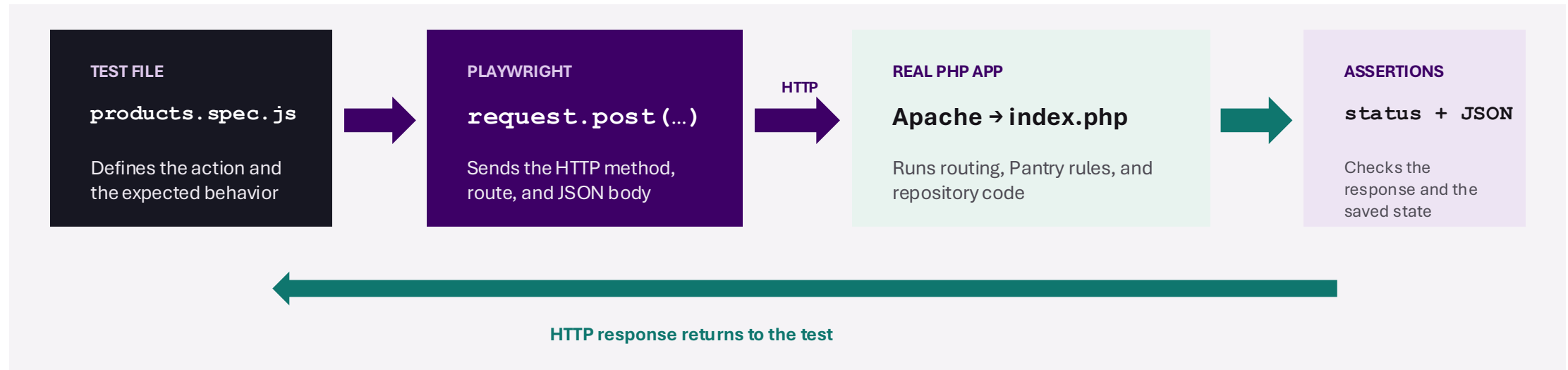
- **NEEDED TESTS: Using more than is in stock is refused, and the amount does not change**
- **Five parts of a prompt that gets a usable test**
  - 1. The code under test — paste Stock.php, whole
  - 2. An existing test file, for style
  - 3. The behavior, as a sentence with exact values
  - 4. What not to do — no editing src, no assertTrue(true)
- **The test must be RED before you fix anything**
  - Green against code you know is wrong means it is testing the wrong thing. Read what came back.
- **The model tests what the code does. Only you can say what it should do.**

# API Tests Exercise the App Through HTTP via Playwright

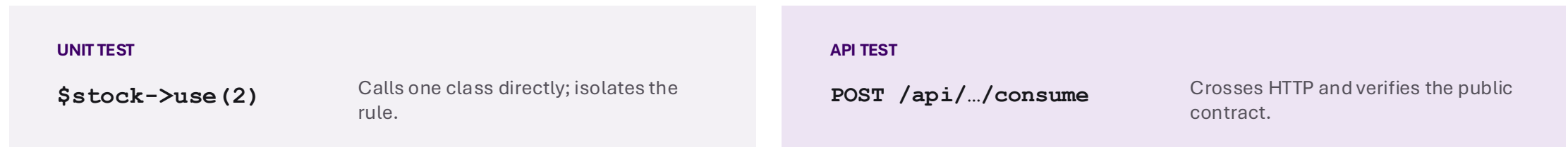
Playwright can call the API directly—no browser UI—so the test sees the same contract as any other client.

HTTP • NO UI

## THE REQUEST-RESPONSE BOUNDARY



## SAME BEHAVIOR, WIDER BOUNDARY



Unit tests ask whether the rule works. API tests ask whether the application exposes that rule correctly.

# End-to-End Testing (Playwright)

- Slower (200–300 ms each) — a real Chromium browser
- From `tests/e2e/pantry.spec.js`:

```
await page.goto('/');  
await page.fill('#new-product input[name="name"]', name);  
await page.click('#new-product button[type="submit"]');  
  
const row = page.locator('#products tr', { hasText: name });  
await expect(row.locator('.amount')).toHaveText('0');  
await expect(row.locator('.badge')).toHaveText('fresh');
```

# One Rule, Three Tests

- **“You cannot use more than you have”**

- Found by hand in Part 0: use 3 rice from a shelf holding 0 → minus three, and no message

- **Unit — the class refuses**

```
$this->expectException(InsufficientStockException::class);    $stock->consume(5);
```

- **API — the service refuses, and says how**

```
expect(res.status()).toBe(422);    expect(product.amount).toBe(1);
```

- **Browser — the person sees it**

```
await expect(page.locator('.err')).toHaveText('Not enough in stock');
```

- One three-line fix in `Stock::consume()` turns all three green. `index.php` never changed — the rule lives in one place, so it is enforced everywhere.



# Running Things — Three Commands,

- **One image, one file, three verbs**

`docker compose watch`                      # the app at `http://localhost:8080`; edits sync in  
as you save

`docker compose run --rm unit`            # PHPUnit            `tests/unit`            one-shot: does  
the job, exits

`docker compose run --rm api`            # Playwright        `tests/api`        (no browser)

`docker compose run --rm e2e`            # Playwright        `tests/e2e`        (Chromium)

`docker compose down`                    # stop                (-v wipes the data volume too)

# What Test Output Looks Like

- **PHPUnit — tests/unit (--testdox: every test as its sentence)**

```
..... 21 / 21 (100%)  
Stock  
✓ A new stock is empty    ✓ Buying adds to the amount    ✓ Using all of it leaves zero    ...  
OK (21 tests, 24 assertions)    Time: 00:00.005
```

- **Playwright — tests/api (as shipped: one is red on purpose)**

```
X 5 consuming more than is in stock is refused with 422 (10ms)  
    Expected: 422    Received: 200  
1 failed, 5 passed
```

- **Playwright — tests/e2e**

```
✓ a user can add a product and see it in the table (217ms)  
✓ buying and using change the amount on the shelf (317ms)
```

- Twenty-one unit tests in five milliseconds. One browser test in a quarter of a second. That ratio is the pyramid.

# Exercise — Five Parts (today)

- **0. Use it**
  - docker compose watch. Add Milk, Eggs, Rice. Use 3 rice from a shelf holding 0. Write down what looks wrong — that is question 1.
- **1. Read**
  - StockTest and DueDateTest before Stock.php. Run. Break add(). Read the failure.
- **2. Prompt**
  - Write the five-part prompt, compare what comes back to the reference, make sure it is RED, then fix consume(). Prompt and output — that is question 2.
- **3. API, then 4. Browser**
  - The 422 test is green now. Write one more test at each level and mutation-check it.
- **Hand in — before you leave**
  - inclass/wk3/ANSWERS.md, two short questions, committed and pushed to main before the end of class.

# Key Takeaways

- **Use all 3 testing layers**
  - Unit tests catch logic bugs fast; integration tests verify APIs; E2E tests catch real user-facing bugs
- **Balance speed vs realism**
  - More unit tests (fast), fewer E2E tests (slow but realistic)
- **Automate testing in your workflow**
  - CI/CD runs tests on every push - catch bugs before production
- **Tests document expected behavior**
  - If the tests pass, the API contract is met